

Improving Test Execution Time with Improved Cache Locality

Panagiotis Stratis
School of Informatics
University of Edinburgh
Edinburgh, United Kingdom
s1329012@sms.ed.ac.uk

ICSE Theme-Software testing

Keywords-Cache misses, Instruction locality, Testing

I. INTRODUCTION

As software systems become more complex, the number of test cases required for effective testing becomes intractable. Furthermore, modern agile development methodologies [2] encourage the execution of test suites as often as possible in an attempt to minimize the time between the introduction and identification of bugs. However, repeatedly running a large test suite can affect the software schedule [3]. This is more apparent in safety critical programs since they are typically associated with very large test suites [8].

Cache misses have been identified as a major factor that affects software execution time. The efficient use of cache memory is essential for minimizing the data access latency during program execution and, therefore, for improving its execution time. Cache misses are reduced by increasing the locality of memory references for both instructions and data.

In our current work we target the instruction locality problem in the context of testing. Existing compiler work considers improving locality over *single* program runs. In the context of testing, however, the same program is executed multiple times. In this paper, we propose a novel approach for leveraging instruction locality *across* the execution of test cases. We **permute** the test cases so that subsequent test case runs share as many instructions as possible. Optimising for instruction locality *across* test cases is entirely novel.

II. BACKGROUND AND RELATED WORK

Test case selection/prioritization [13] and test suite reduction [12] optimization techniques have been proposed by the software testing community for reducing the size of test suites and, as a result, improve their execution time. The main idea behind these approaches is to utilize a metric (e.g. branch coverage) in order to execute as **few** test cases as possible. However, reducing the number of test cases is known to reduce the test suite's fault-finding capability [6]. Furthermore, real-world test suites tend to be very large even after these optimizations. Our approach aims to reduce the execution time of test suites by permuting their test cases, not removing them and can be applied to already minimized test suites.

For improving spatial locality of instructions compiler engineers have proposed code layout optimizations [5] and procedure/function re-ordering [7]. On the other hand, temporal instruction locality has not been explored since existing optimizations target single program executions where the probability of similar instruction sequences is minimal¹. In the context of testing however, as the number of test cases increases, the probability of encountering clusters of test cases that execute similar instruction sequences increases as well. Our approach aims to increase instruction temporal locality *across* the execution of test cases.

III. OUR APPROACH

For enhancing temporal locality of instructions *across* test case executions we need the following:

- 1) A definition of distance between two test cases.
- 2) An algorithm which, given the test case distances, will be able to compute a permutation in which the distance between subsequent test cases is minimized.

We define our distance metric based on the idea that instruction locality between two test case runs will be proportional to the number of their common instructions. Therefore, we define our distance metric to be the number of different instructions² between two test cases:

$$D(T_i, T_j) = \# \text{ different instructions of } T_i \text{ and } T_j \quad (1)$$

Having the test case distances in place, the task of computing our permutation is similar to the problem of least cost Hamiltonian path which is known to be NP-hard. In our work, we utilize the nearest neighbour analysis [9], a well known approximation algorithm. Our methodology consists of two steps: We first dynamically analyse the test cases of the subject program in order to map them to the set of their executed instructions. We then produce the permutation using the nearest neighbour algorithm. Our approach is illustrated in figure 1.

¹Unless the instructions are part of a loop, in which case existing loop transformations [4] will help improve locality.

²In our present work we consider source code instructions.

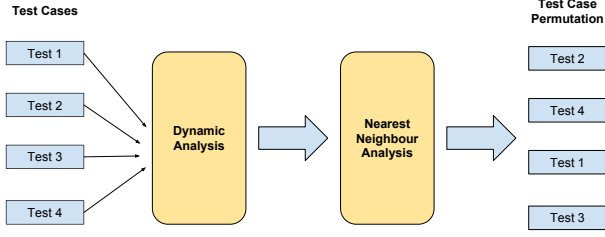


Fig. 1. Our approach

IV. EXPERIMENT AND RESULTS

In our evaluation we use 4 subject programs from the SIR repository [10] and attempt to answer the following questions:

- **Q 1.** Does the order of which the test cases are executed have an impact on the test suite's execution time?
- **Q 2.** How does our optimised permutation compare to random permutations?
- **Q 3.** What is the complexity of our optimization?
- **Q 4.** How does our optimization perform against existing test adequacy measures?

For answering **Q1** we produced 2000 random test case permutations for each subject program and measured their execution time. It is clear from figure 2 that the order of tests in a test suite **affects** the overall execution time. The magnitude of the effect varies across programs because it is related to the program features as well as the nature of the associated test suite. The differences between the best and worst permutation execution time ranged between 15.18% and 34.46%.

Q2 is answered by measuring the execution time of our optimized permutation and compare it against the random permutations. In all subject programs the **gain** of our optimized permutation was in the range of 2.73% to 7.97% when compared against the average of the random permutations and in the range of 11.88% to 26.59% when compared against the worst permutation.

Our optimization is exponential to the number of test cases in the test suite. We, therefore, implemented an *approximate* nearest neighbour algorithm by using a technique called Locality Sensitive Hashing [1] and measured the execution time of the approximated permutation as well in order to fully answer **Q3**. The approximated permutation was only 0.39% to 1.36% **slower** in all benchmarks.

Finally, we answer **Q4** by comparing our approach against a branch coverage permutation. We picked branch coverage because is one of the most popular measures of test adequacy and it is commonly used in test case prioritisation [11]. The branch coverage permutation was **slower** than our optimised permutation (loss ranging from 7.34% to 17.51%) and the random average (loss ranging from 0.76% to 2.66%) in all subject programs. This can be explained from the fact that in branch coverage every subsequent test case should execute as many new branches as possible and, therefore, as many new instructions as possible.

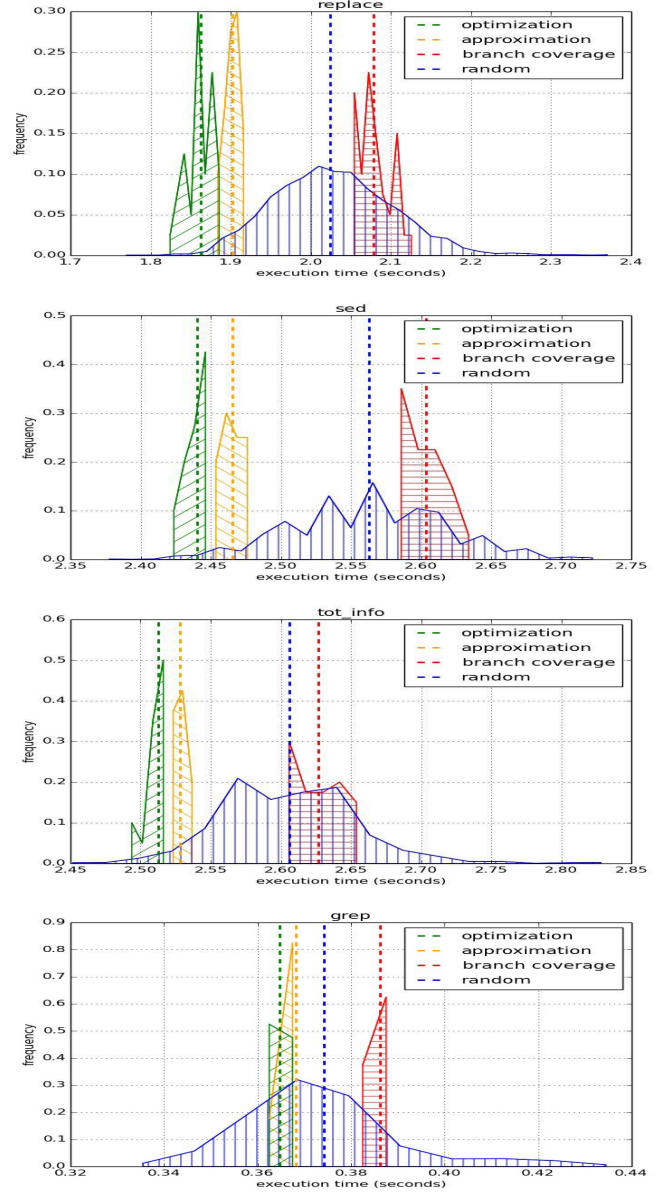


Fig. 2. SIR benchmarks test suite executions

V. FUTURE WORK

Our future work will address the following topics:

- **Embedded domain** - We intent to apply our approach to embedded applications which are known to be CPU intensive and typically are associated with a large number of test cases.
- **Overhead** - For each subject program we are going to explore how our optimization and approximation algorithms scale with respect to the number of test cases.
- **Cache** - We plan to profile the executions of each test case order and explore the relationship between instruction cache behaviour and execution time.

REFERENCES

- [1] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*, pages 459–468. IEEE, 2006.
- [2] Kent Beck, Mike Beedle, Arie Van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. Manifesto for agile software development. 2001.
- [3] Barry Boehm, Chris Abts, and Sunita Chulani. Software development cost estimation approaches a survey. *Annals of software engineering*, 10(1-4):177–205, 2000.
- [4] Steve Carr, Kathryn S McKinley, and Chau-Wen Tseng. *Compiler optimizations for improving data locality*, volume 29. ACM, 1994.
- [5] J Bradley Chen and Bradley DD Leupen. Improving instruction locality with just-in-time code layout. In *Proceedings of the USENIX Windows NT Workshop*, pages 25–32, 1997.
- [6] Mats PE Heimdahl and Devaraj George. Test-suite reduction for model based tests: Effects on test quality and implications for testing. In *Proceedings of the 19th IEEE international conference on Automated software engineering*, pages 176–185. IEEE Computer Society, 2004.
- [7] W-m W Hwu and Pohua P Chang. Achieving high instruction cache performance with an optimizing compiler. In *ACM SIGARCH Computer Architecture News*, volume 17, pages 242–251. ACM, 1989.
- [8] John C Knight. Safety critical systems: challenges and directions. In *Software Engineering, 2002. ICSE 2002. Proceedings of the 24rd International Conference on*, pages 547–550. IEEE, 2002.
- [9] Shen Lin and Brian W Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations research*, 21(2):498–516, 1973.
- [10] Gregg Rothmel, Sebastian Elbaum, Alex Kinnear, and Hyunsook Do. Software-artifact infrastructure repository. *UR L* <http://sir.unl.edu/portal>, 2006.
- [11] Gregg Rothmel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on software engineering*, 27(10):929–948, 2001.
- [12] Patrick J Schroeder and Bogdan Korel. *Black-box test reduction using input-output analysis*, volume 25. ACM, 2000.
- [13] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2):67–120, 2012.